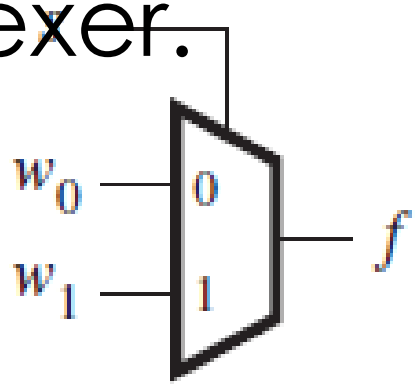
Multiplexers

A multiplexer circuit has a number of data inputs, one or more select inputs, and one output. It passes the signal value on one of the data inputs to the output. The data input is selected by the values of the select inputs.

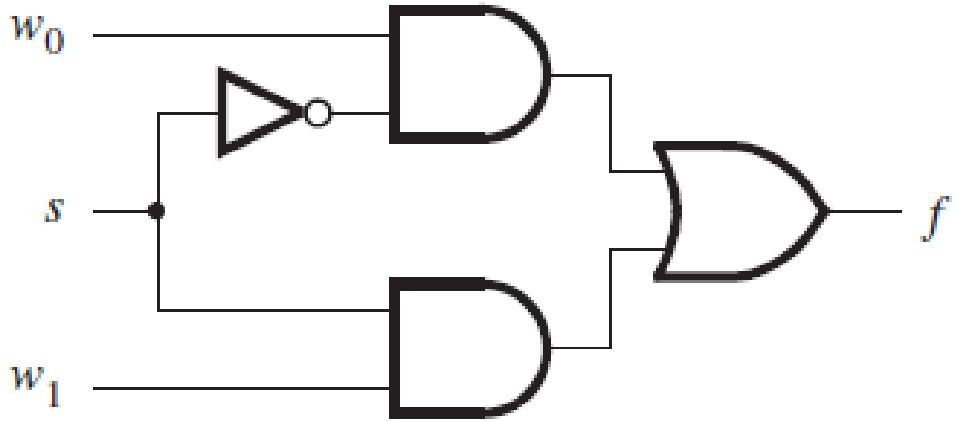
2-to-1 multiplexer.



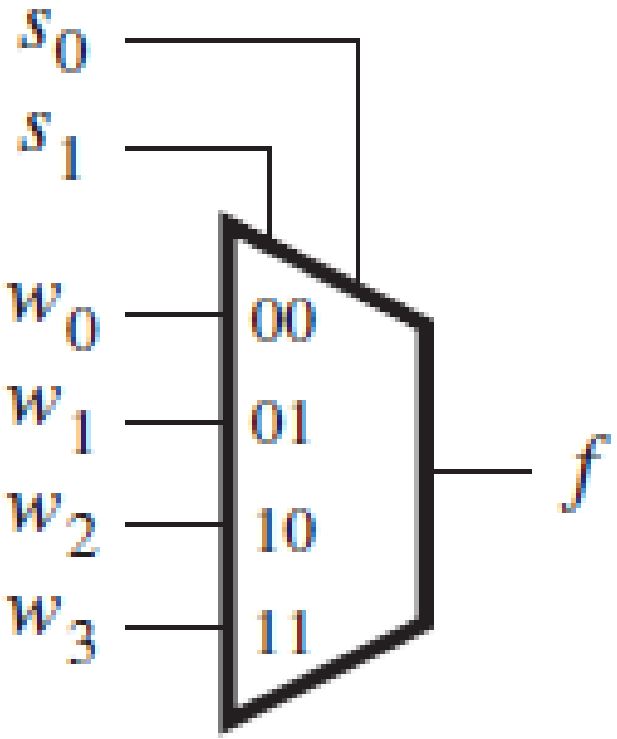
(a) Graphical symbol

s	f
0	w_0
1	w_1

(b) Truth table



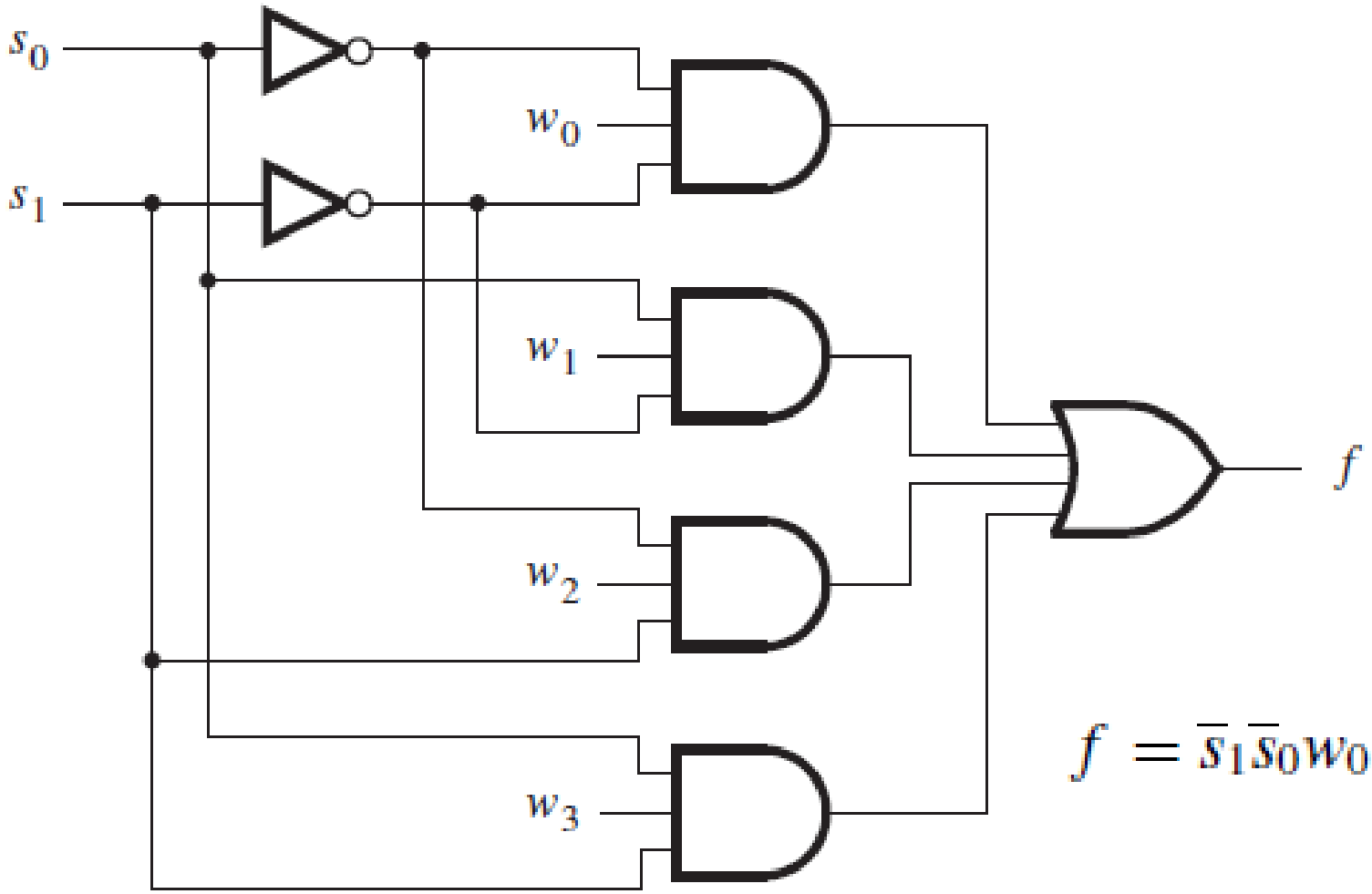
4-to-1 multiplexer.



(a) Graphical symbol

s_1	s_0	f
0	0	w_0
0	1	w_1
1	0	w_2
1	1	w_3

(b) Truth table



s_1	s_0	f
0	0	w_0
0	1	w_1
1	0	w_2
1	1	w_3

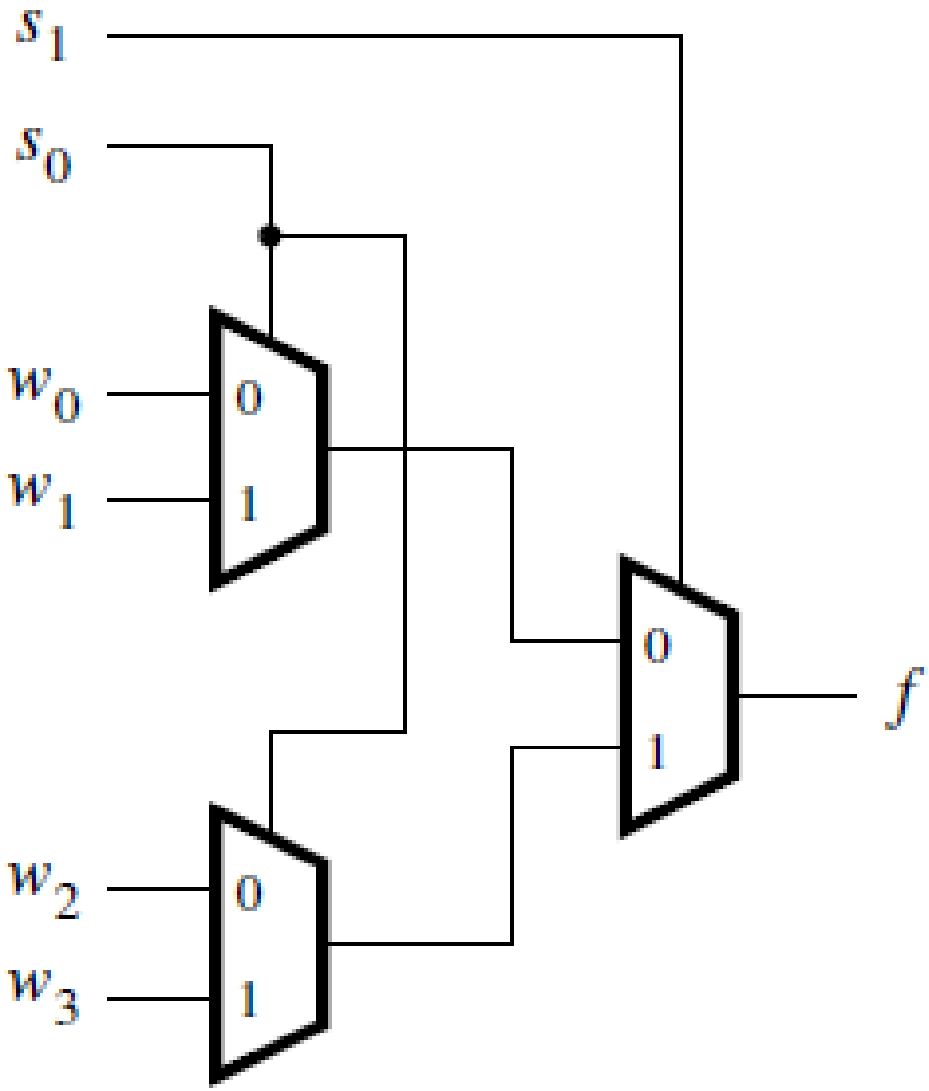
$$f = \overline{s}_1\overline{s}_0 w_0 + \overline{s}_1 s_0 w_1 + s_1\overline{s}_0 w_2 + s_1 s_0 w_3$$

(c) Circuit

Usually, the number of data inputs, n, is an integer power of two. A multiplexer that has n data inputs, $w0, \ldots, wn-1$, requires $\lceil \log_2 n \rceil$ select inputs and it has one output.

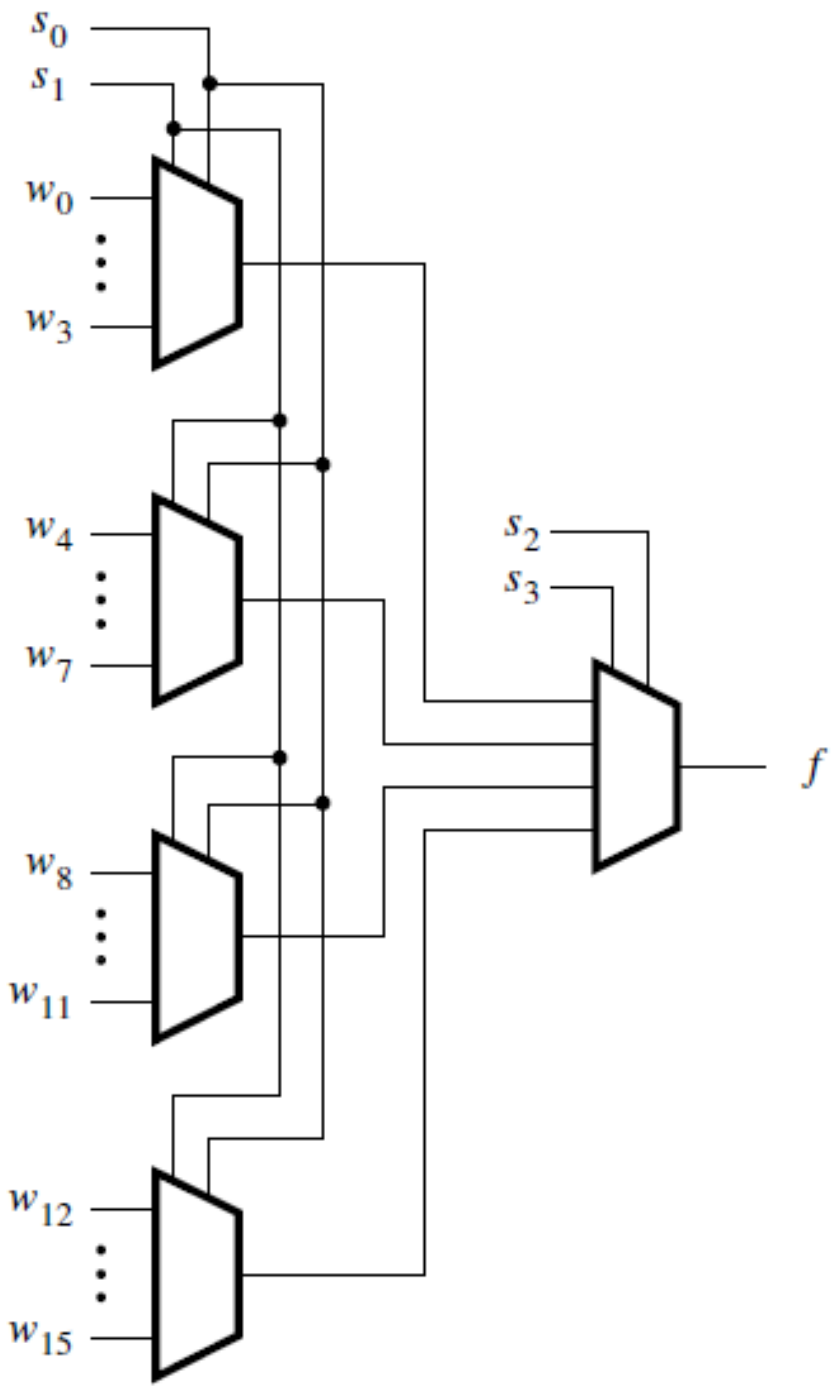
Larger multiplexers can also be constructed from smaller multiplexers. For example, the 4-to-1 multiplexer can be built using three 2-to-1 multiplexers as shown below:

s_1	s_0	f
0	0	w_0
0	1	w_1
1	0	w_2
1	1	w_3



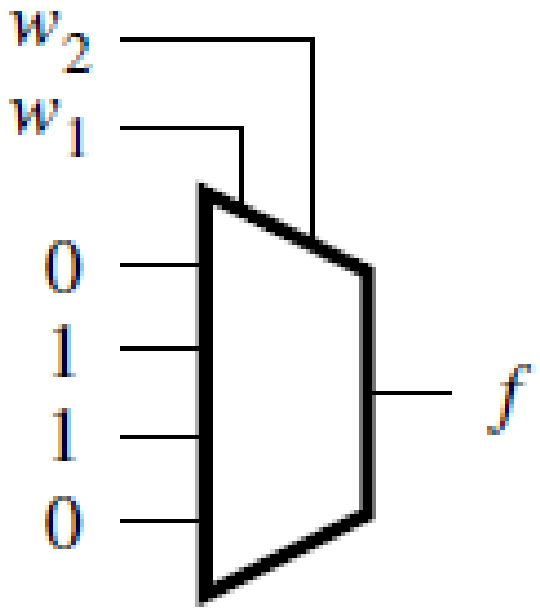
Using 2-to-1 multiplexers to build a 4-to-1 multiplexer.

Similarly, 16-to-1 multiplexer can be constructed with five 4-to-1 multiplexers.



Synthesis of Logic Functions Using Multiplexers

w_1	w_2	f
0	0	0
0	1	1
1	0	1
1	1	0

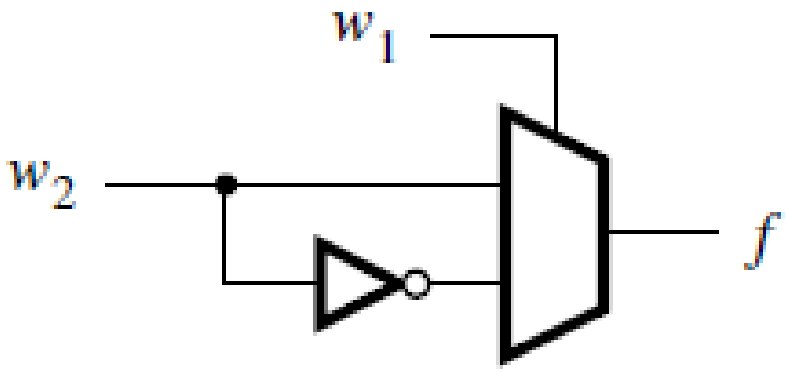


(a) Implementation using a 4-to-1 multiplexer

Example 1 : Two input XOR function using 2 to 1 multiplexer

w_1	w_2	f
0	0	0
0	1	1
1	0	1
1	1	0

w_1	f
0	w_2
1	$\overline{w}_2$

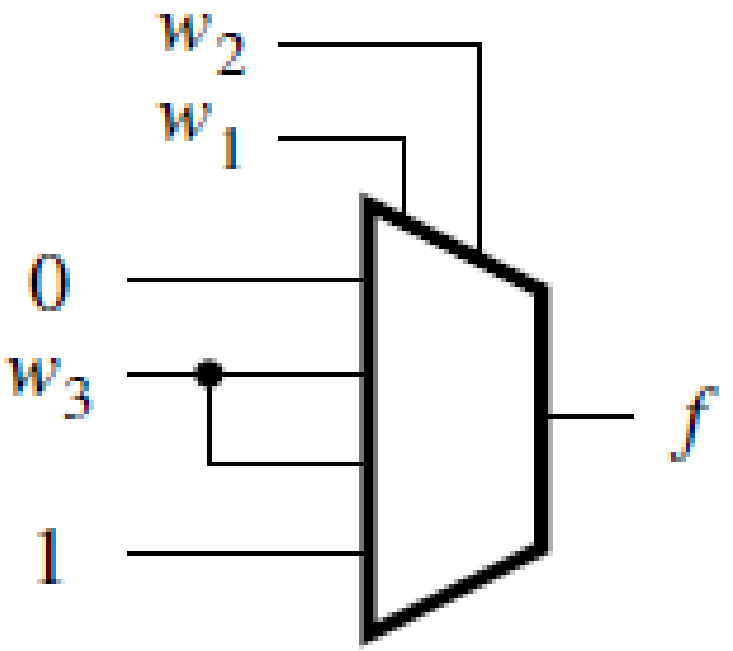


(c) Circuit

Example 2 : Three input majority function using 4 to 1 multiplexer

w_1	w_2	w_3	f
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

w_1	w_2	f
0	0	0
0	1	w_3
1	0	w_3
1	1	1

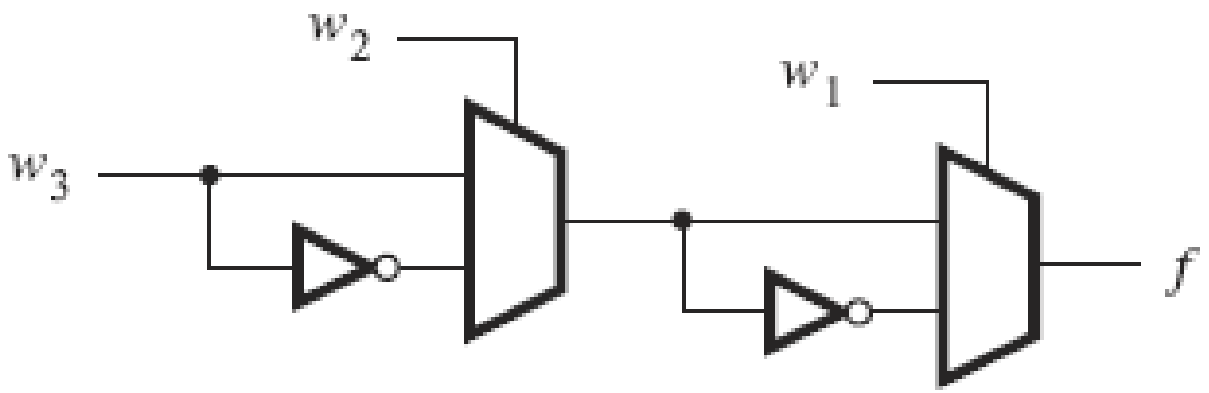


(b) Circuit

Example 3: Implement a three-input XOR gate using 2-to-1 multiplexer.

w_1	w_2	w_3	f	
0	0	0	0	$w_2 \oplus w_3$
0	0	1	1	
0	1	0	1	
0	1	1	0	
1	0	0	1	$\overline{w_2 \oplus w_3}$
1	0	1	0	
1	1	0	0	
1	1	1	1	

(a) Truth table

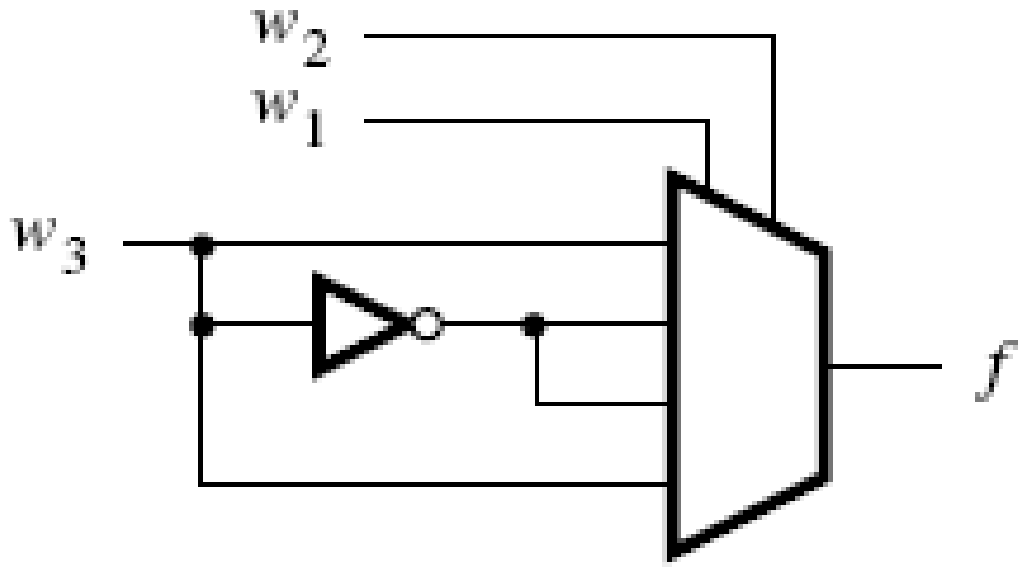


(b) Circuit

Example 4: Implement a three-input XOR gate using 4-to-1 multiplexer.

w_1	w_2	w_3	f	
0	0	0	0	w_3
0	0	1	1	
0	1	0	1	$\overline{w}_3$
0	1	1	0	
1	0	0	1	$\overline{w}_3$
1	0	1	0	
1	1	0	0	w_3
1	1	1	1	

(a) Truth table



(b) Circuit

Multiplexer Synthesis Using Shannon's Expansion

The inputs to the multiplexers are the constants 0 and 1, or some variable or its complement. Besides using such simple inputs, it is possible to connect more complex circuits as inputs to a multiplexer, allowing functions to be synthesized using a combination of multiplexers and other logic gates.

Example 5 : Three input majority function using 2 to 1 multiplexer

w_1	w_2	w_3	f
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

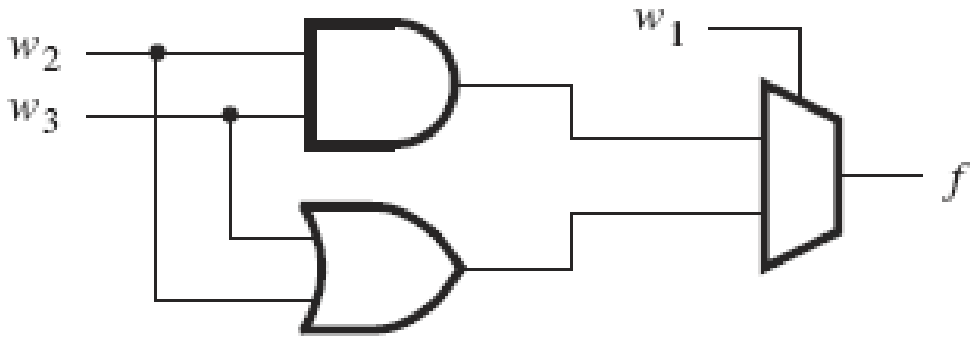
(a) Truth table

-

w_2w_3

w_2+w_3

w_1	f
0	w_2w_3
1	$w_2 + w_3$



(b) Circuit

Shannon's Expansion

w_1	w_2	w_3	f
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

This implementation can be derived using algebraic manipulation as follows. The function is expressed in sum-of-products form as

$$f = \overline{w}_1 w_2 w_3 + w_1 \overline{w}_2 w_3 + w_1 w_2 \overline{w}_3 + w_1 w_2 w_3$$

It can be manipulated into

$$\begin{aligned} f &= \overline{w}_1 (w_2 w_3) + w_1 (\overline{w}_2 w_3 + w_2 \overline{w}_3 + w_2 w_3) \\ &= \overline{w}_1 (w_2 w_3) + w_1 (w_2 + w_3) \end{aligned}$$

Multiplexer implementations of logic functions require that a given function be decomposed in terms of the variables that are used as the select inputs. This can be accomplished by means of a theorem proposed by Claude Shannon

Shannon's Expansion Theorem Any Boolean function $f(w_1, \ldots, w_n)$ can be written in the form

$$f(w_1, w_2, \ldots, w_n) = \overline{w}_1 \cdot f(0, w_2, \ldots, w_n) + w_1 \cdot f(1, w_2, \ldots, w_n)$$

In Shannon's expansion the term $f(0, w_2, \ldots, w_n)$ is called the *cofactor* of f with respect to $\overline{w}_1$; it is denoted in shorthand notation as $f_{\overline{w}_1}$. Similarly, the term $f(1, w_2, \ldots, w_n)$ is called the cofactor of f with respect to w_1, written f_{w_1}. Hence we can write

$$f = \overline{w}_1 f_{\overline{w}_1} + w_1 f_{w_1}$$

In general, if the expansion is done with respect to variable w_i, then $f_{\overline{w}_i}$ denotes $f(w_1, \ldots, w_{i-1}, 0, w_{i+1}, \ldots, w_n)$, f_{w_i} denotes $f(w_1, \ldots, w_{i-1}, 1, w_{i+1}, \ldots, w_n)$, and

$$f(w_1, \ldots, w_n) = \overline{w}_i f_{\overline{w}_i} + w_i f_{w_i}$$

Example 1

$$f(w_1, w_2, w_3) = w_1w_2 + w_1w_3 + w_2w_3$$

Expanding this function in terms of w_1 gives

$$\begin{aligned} f &= \overline{w}_1(0 \cdot w_2 + 0 \cdot w_3 + w_2w_3) + w_1(1 \cdot w_2 + 1 \cdot w_3 + w_2w_3) \\ &= \overline{w}_1(w_2w_3) + w_1(w_2 + w_3) \end{aligned}$$

Example 2

For the three-input XOR function, we have

$$
\begin{aligned}
f &= w_1 \oplus w_2 \oplus w_3 \\
&= \overline{w}_1(0 \oplus w_2 \oplus w_3) + w_1(1 \oplus w_2 \oplus w_3) \\
&= \overline{w}_1 \cdot (w_2 \oplus w_3) + w_1 \cdot \overline{(w_2 \oplus w_3)}
\end{aligned}
$$

Example 3

$$f = \overline{w}_1 w_3 + w_2 \overline{w}_3$$

Decomposition using w_1 gives

$$\begin{aligned} f &= \overline{w}_1 f_{\overline{w}_1} + w_1 f_{w_1} \\ &= \overline{w}_1 (w_3 + w_2) + w_1 (w_2 \overline{w}_3) \end{aligned}$$

Decomposition using w_3 gives

$$\begin{aligned} f &= \overline{w}_3 f_{\overline{w}_3} + w_3 f_{w_3} \\ &= \overline{w}_3 (w_2) + w_3 (\overline{w}_1) \end{aligned}$$

Using w_2 instead of w_1 gives

$$\begin{aligned} f &= \overline{w}_2 f_{\overline{w}_2} + w_2 f_{w_2} \\ &= \overline{w}_2 (\overline{w}_1 w_3) + w_2 (\overline{w}_1 w_3 + \overline{w}_3) \\ &= \overline{w}_2 (\overline{w}_1 w_3) + w_2 (\overline{w}_1 + \overline{w}_3) \end{aligned}$$

Shannon's expansion can be done in terms of more than one variable. For example, expanding a function in terms of w_1 and w_2 gives

$$f(w_1, \ldots, w_n) = \overline{w}_1 \overline{w}_2 \cdot f(0, 0, w_3, \ldots, w_n) + \overline{w}_1 w_2 \cdot f(0, 1, w_3, \ldots, w_n)$$
$$+ w_1 \overline{w}_2 \cdot f(1, 0, w_3, \ldots, w_n) + w_1 w_2 \cdot f(1, 1, w_3, \ldots, w_n)$$

Example 4: Implement the following function using

i) 2-to-1 multiplexers

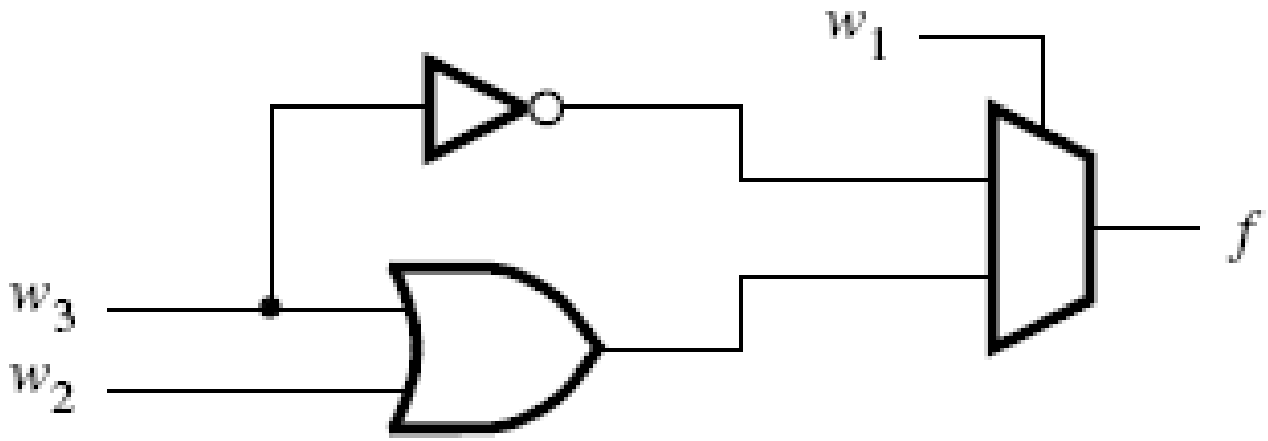
ii) 4-to-1 multiplexers

and any other necessary gates

$$f = \overline{w}_1\overline{w}_3 + w_1w_2 + w_1w_3$$

Shannon's expansion using w_1 gives

$$\begin{aligned} f &= \overline{w}_1 f_{\overline{w}_1} + w_1 f_{w_1} \\ &= \overline{w}_1(\overline{w}_3) + w_1(w_2 + w_3) \end{aligned}$$

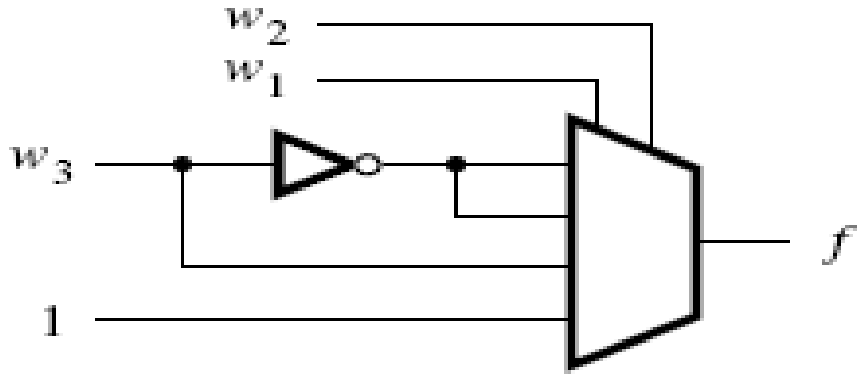


(a) Using a 2-to-1 multiplexer

Example 4 (Contd…)

Shannon's expansion using both w_1 and w_2 gives

$$f = \overline{w}_1\overline{w}_2 f_{\overline{w}_1\overline{w}_2} + \overline{w}_1 w_2 f_{\overline{w}_1 w_2} + w_1\overline{w}_2 f_{w_1\overline{w}_2} + w_1 w_2 f_{w_1 w_2}$$
$$= \overline{w}_1\overline{w}_2(\overline{w}_3) + \overline{w}_1 w_2(\overline{w}_3) + w_1\overline{w}_2(w_3) + w_1 w_2(1)$$



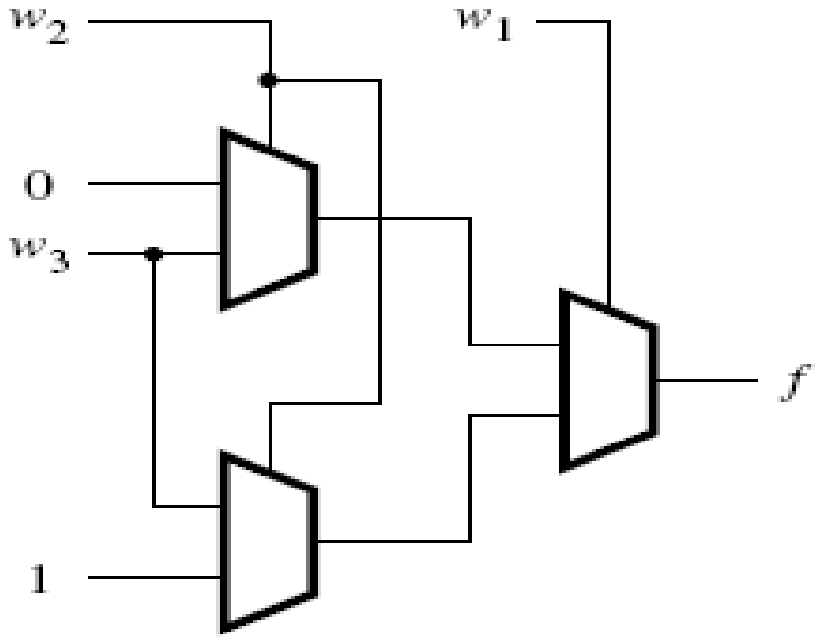
(b) Using a 4-to-1 multiplexer

Example 5: Implement the following function using only 2-to-1 multiplexers

$$f = w_1 w_2 + w_1 w_3 + w_2 w_3$$

Shannon's expansion using w_1 gives

$$f = \overline{w}_1 (w_2 w_3) + w_1 (w_2 + w_3 + w_2 w_3)$$
$$= \overline{w}_1 (w_2 w_3) + w_1 (w_2 + w_3)$$



Let $g = w_2 w_3$ and $h = w_2 + w_3$. Expansion of both g and h using w_2 gives

$$g = \overline{w}_2 (0) + w_2 (w_3)$$
$$h = \overline{w}_2 (w_3) + w_2 (1)$$

Consider the function $f = \overline{w}_1\overline{w}_3 + w_2\overline{w}_3 + \overline{w}_1 w_2$. Use the truth table to derive a circuit for f that uses a 2-to-1 multiplexer.

Repeat Problem 4.3 for the function $f = \overline{w}_2\overline{w}_3 + w_1 w_2$.

For the function $f(w_1, w_2, w_3) = \sum m(0, 2, 3, 6)$, use Shannon's expansion to derive an implementation using a 2-to-1 multiplexer and any other necessary gates.

Repeat Problem 4.5 for the function $f(w_1, w_2, w_3) = \sum m(0, 4, 6, 7)$.

Decoders

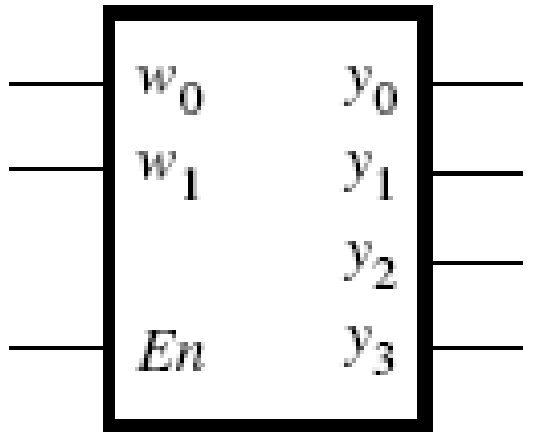
- A binary decoder is a logic circuit with n inputs and 2^n outputs.
- Only one output is asserted at a time, and each output corresponds to one valuation of the inputs.
- The decoder also has an enable input *En*
- If *En*=1, the valuation of $w_{n-1}\ldots w_1 w_0$ determines which of the outputs is asserted.
- If *En*=0, then none of the decoder outputs is asserted.

A k-bit binary code in which exactly one of the bits is set to 1 at a time is referred to as *one-hot encoded*, meaning that the single bit that is set to 1 is deemed to be "hot." The outputs of an enabled binary decoder are one-hot encoded.

2-to-4 Decoders

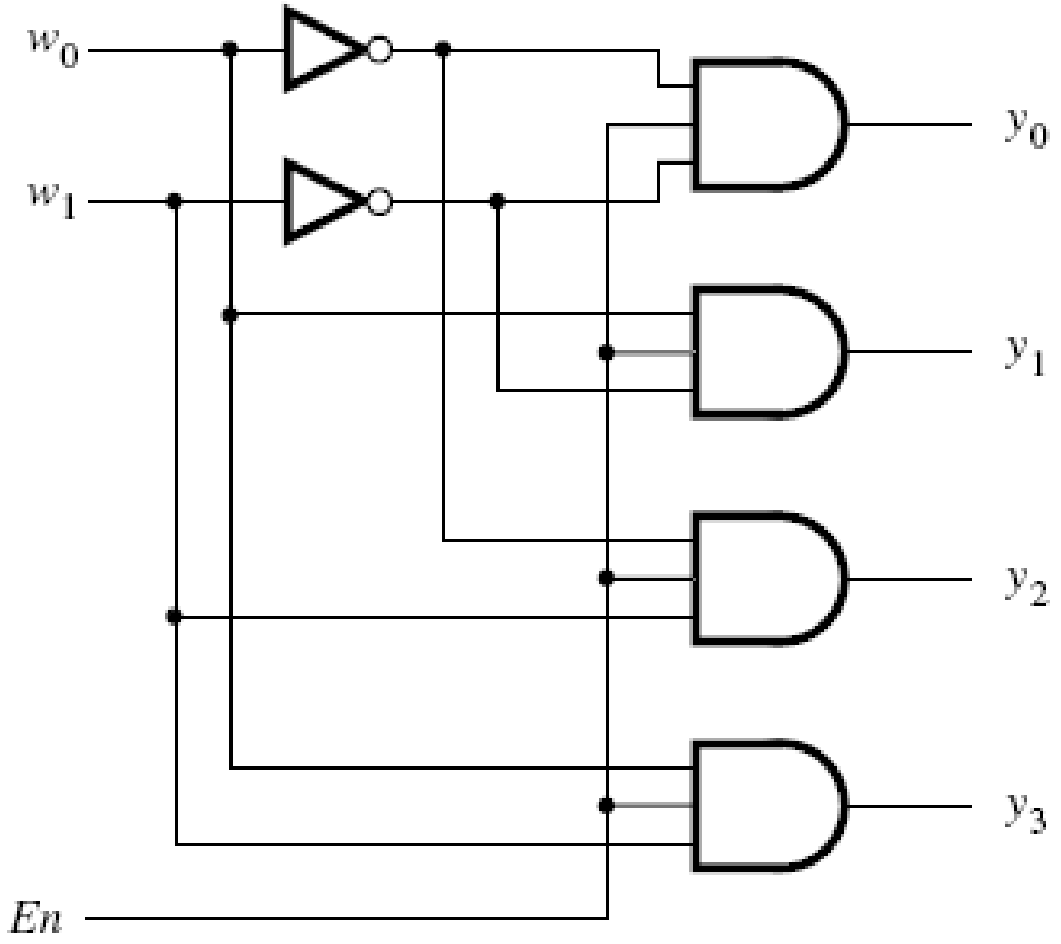
En	w_1	w_0	y_0	y_1	y_2	y_3
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1
0	x	x	0	0	0	0

(a) Truth table



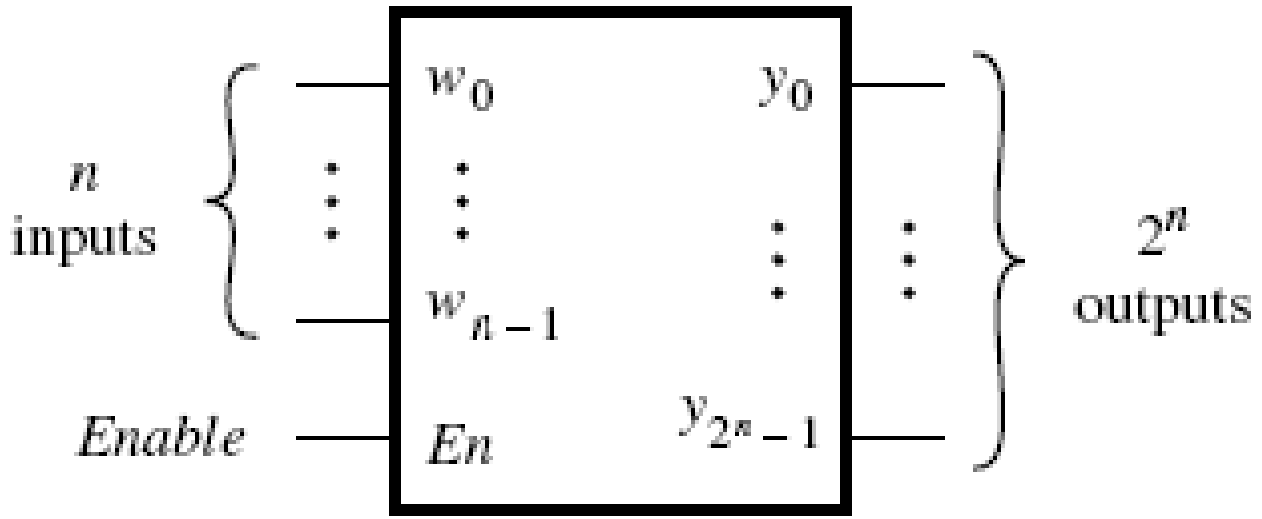
(b) Graphical symbol

2-to-4 Decoders



(c) Logic circuit

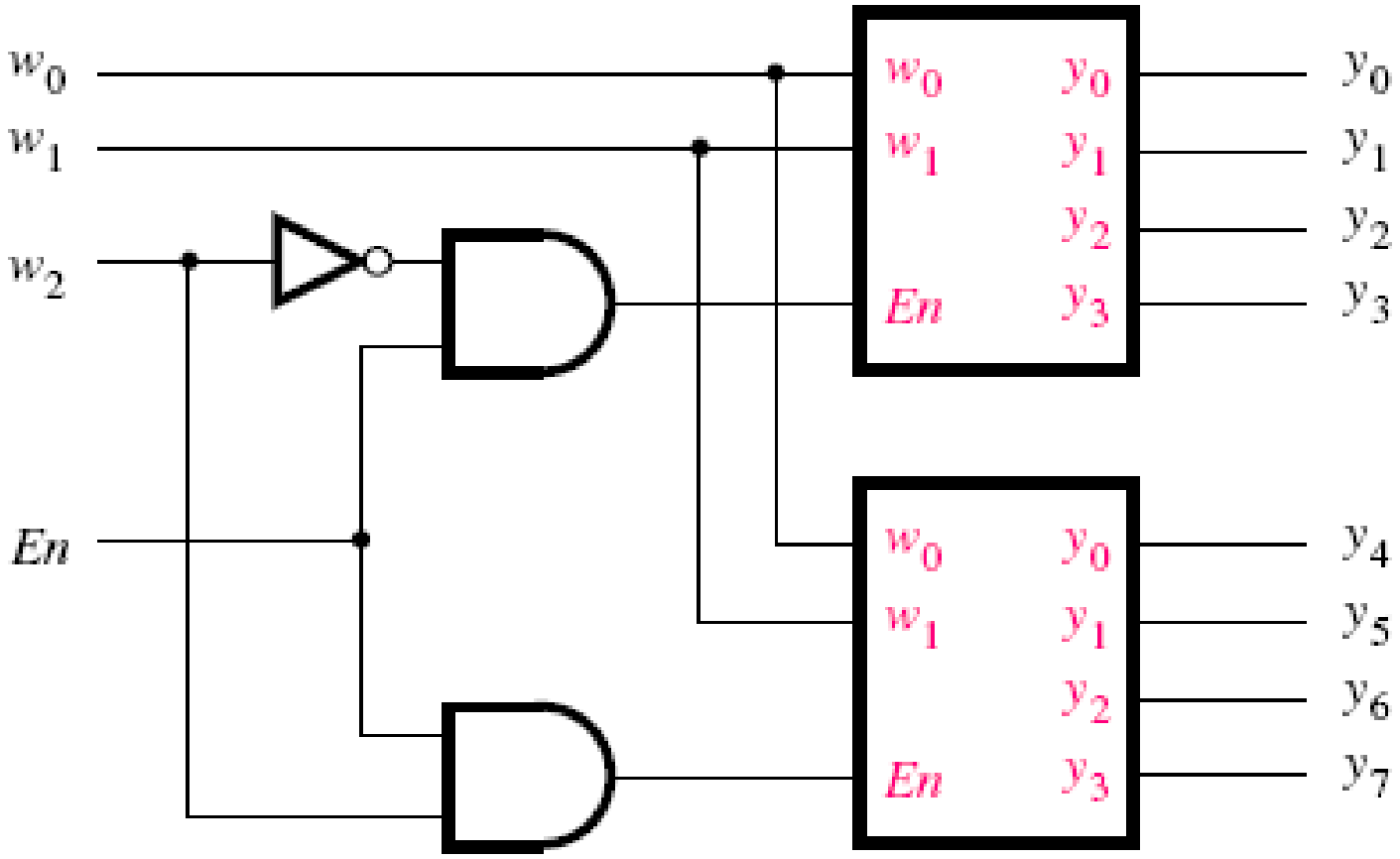
Decoders



(d) An n-to-2^n decoder

Decoders

Larger decoders can be built using the sum-of-products structure or else they can be constructed from smaller decoders.

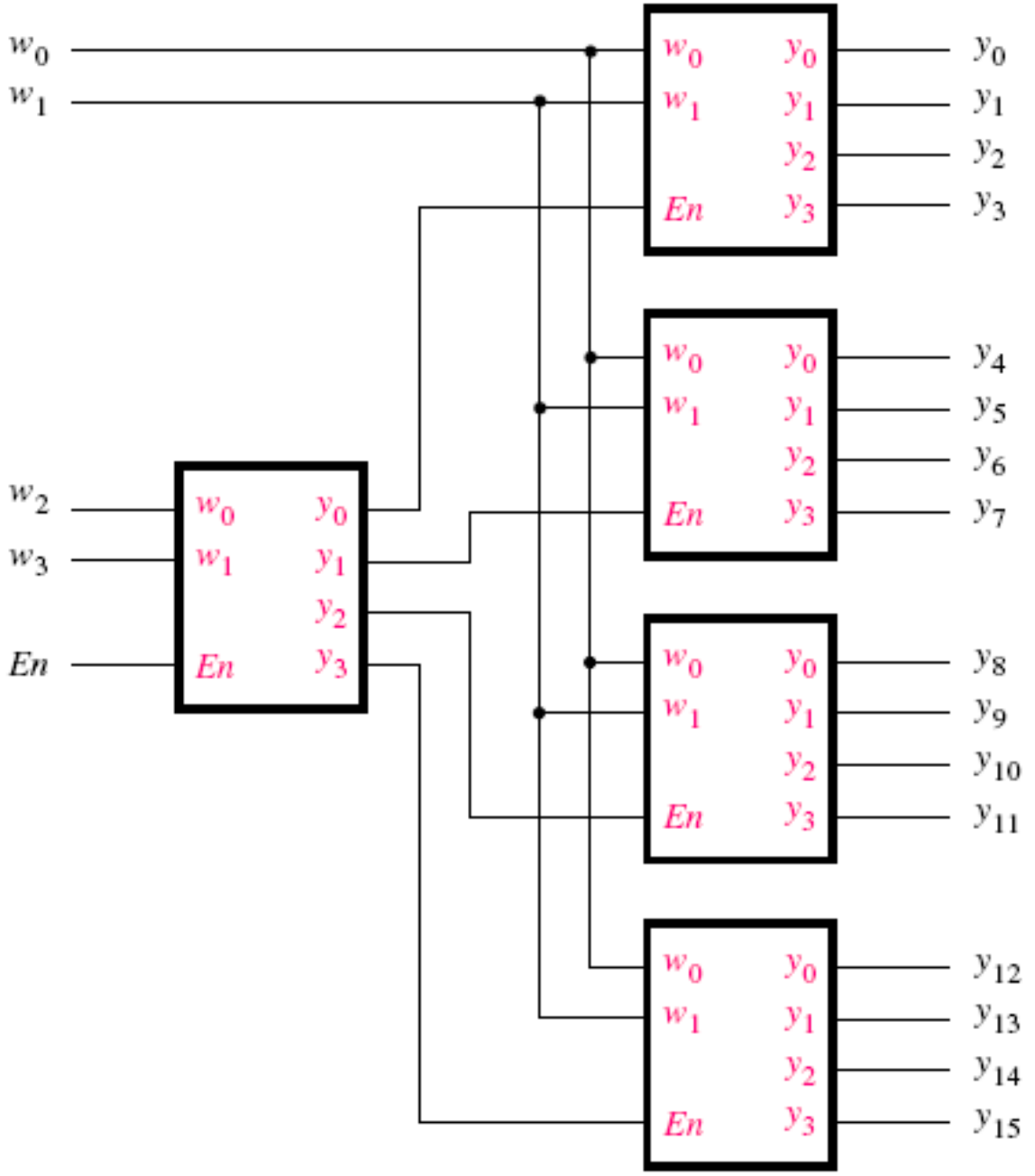


A 3-to-8 decoder using two 2-to-4 decoders.

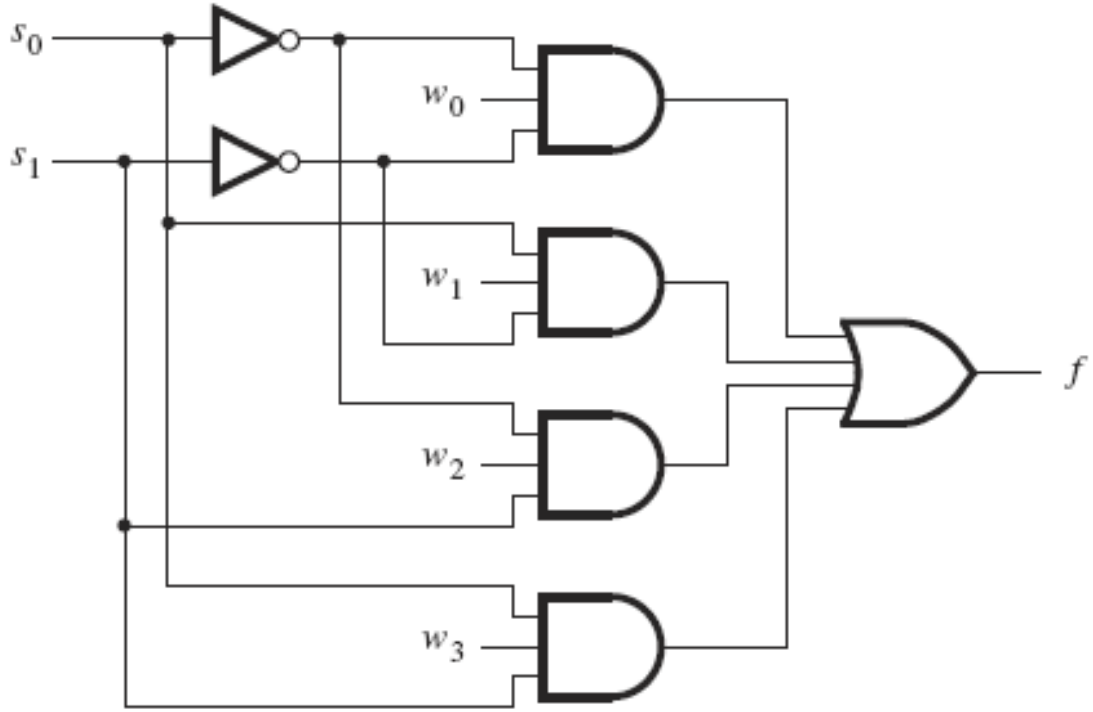
The w2 input drives the enable input of the two decoders.

4-to-16 Decoder

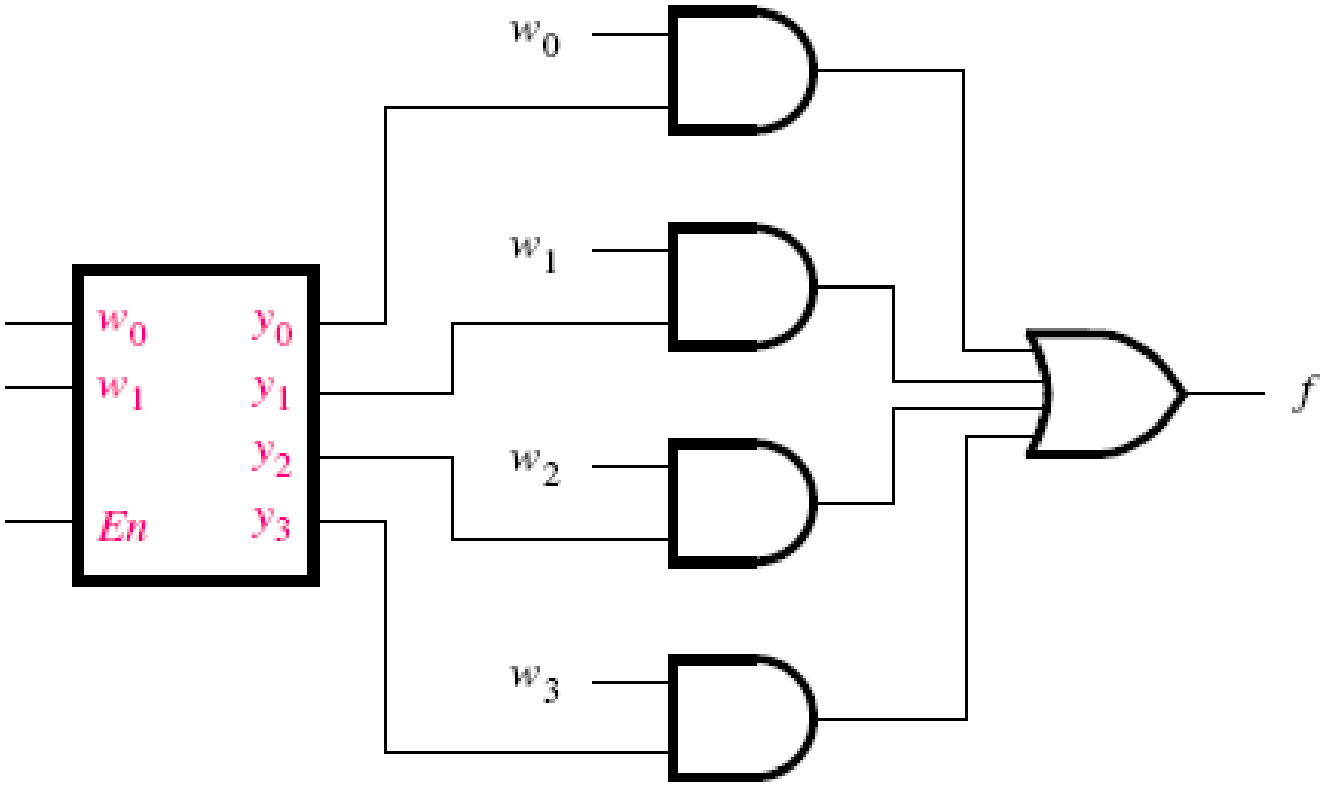
This type of circuit is referred to as a decoder tree.



4-to-1 Multiplexer using Decoder



4-to-1 Multiplexer



4-to-1 Multiplexer using decoder

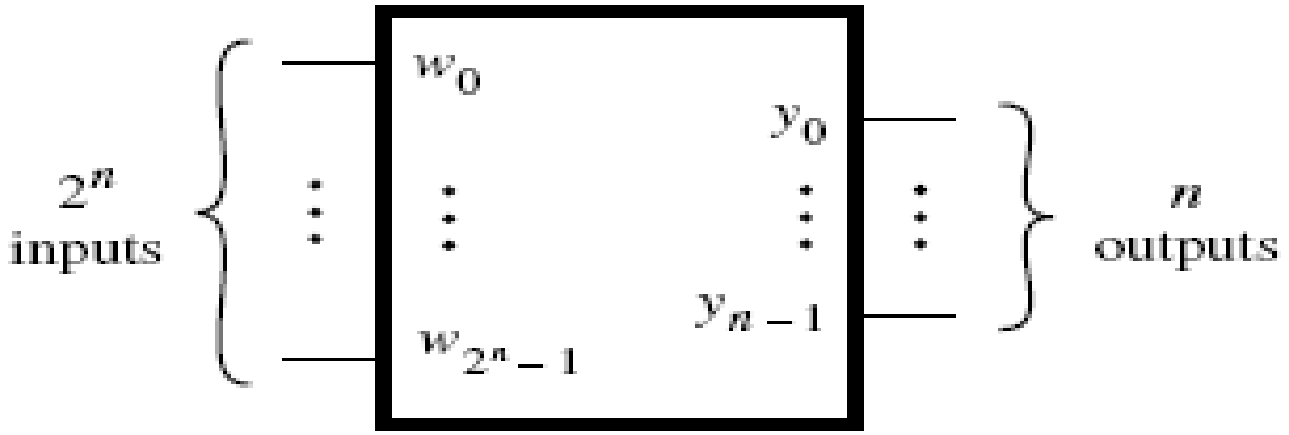
Encoders

An encoder performs the opposite function of a decoder.

Binary Encoders

A *binary encoder* encodes information from 2^n inputs into an n-bit code.

Exactly one of the input signals should have a value of 1, and the outputs present the binary number that identifies which input is equal to 1.

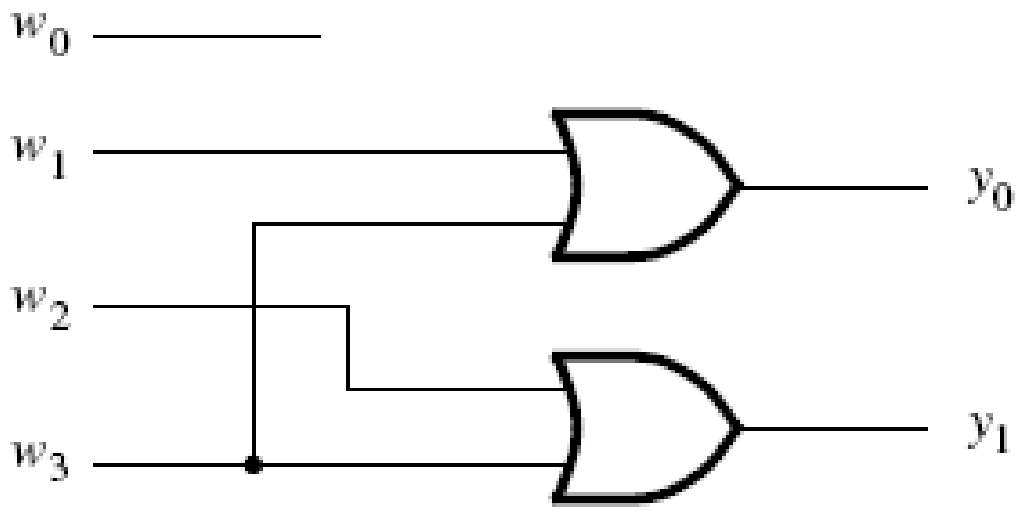


A 2^n-to-n binary encoder.

4-to-2 Binary Encoders

w_3	w_2	w_1	w_0	y_1	y_0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1

(a) Truth table



(b) Circuit

The inputs are one-hot encoded. The input patterns that have multiple inputs set to 1 are treated as don't care conditions.

Encoders are used for transmitting information in a digital system.

Priority Encoders

In a priority encoder each input has a priority level associated with it.
The encoder outputs indicate the active input that has the highest priority.
When an input with high priority is asserted, the inputs with lower priority are ignored.

w_3	w_2	w_1	w_0	y_1	y_0	z
0	0	0	0	d	d	0
0	0	0	1	0	0	1
0	0	1	x	0	1	1
0	1	x	x	1	0	1
1	x	x	x	1	1	1

$$i_0 = \overline{w}_3\overline{w}_2\overline{w}_1 w_0 \qquad y_0 = i_1 + i_3$$

$$i_1 = \overline{w}_3\overline{w}_2 w_1 \qquad y_1 = i_2 + i_3$$

$$i_2 = \overline{w}_3 w_2$$

$$i_3 = w_3$$

$$z = i_0 + i_1 + i_2 + i_3$$

Z is set to 1 when at least one of the inputs is equal to 1.

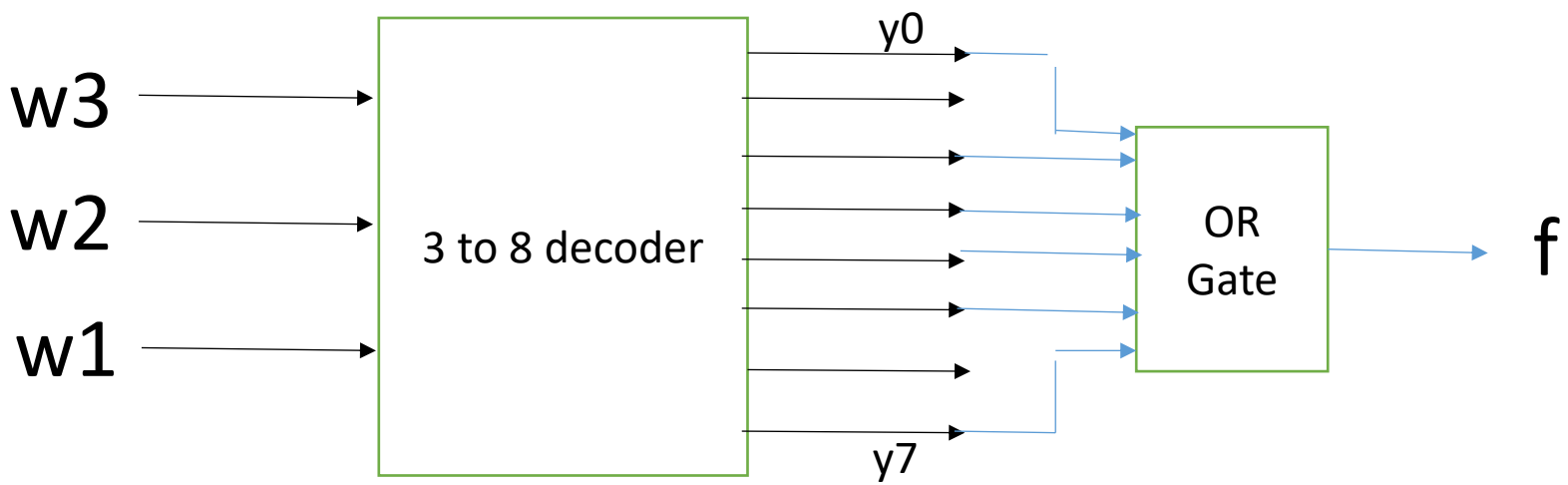
w_0 has the lowest priority and w_3 the highest

Priority encoder having w0 highest priority and w3 least priority

w3	w2	w1	w0	y1	y0	z
x	x	x	1	0	0	1
x	x	1	0	0	1	1
x	1	0	0	1	0	1
1	0	0	0	1	1	1
0	0	0	0	x	x	0

Problems:

1. Show how the function $f(w1, w2, w3) = \sum m(0, 2, 3, 4, 5, 7)$ can be implemented using a 3-to-8 binary decoder and an OR gate.
2. Show how the function $f(w1, w2, w3) = \sum m(1, 2, 3, 5, 6)$ can be implemented using a 3-to-8 binary decoder and an OR gate.


w3
w2
w1
3 to 8 decoder
y0
y7
OR
Gate
f